SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 3- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**25**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	**Debugging Approach & Analysis (20%)**	**Correctness of Debugged Solution (25%)**	**Optimization Techniques Applied (20%)**	**Testing and Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Experiment 1
Debugging Signed Integer Addition Using 2's Complement

Aim
To debug signed integer addition using 2's complement representation, identify arithmetic overflow, and suggest methods to detect and handle overflow correctly.

Problem Statement
A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in an 8-bit system). Analyze the binary computation, determine whether overflow has occurred, and propose an appropriate solution.

Theory
Two's complement is the standard representation for signed integers in computers.

For an 8-bit signed number:

• Minimum value = -128

• Maximum value = +127

If the result of an arithmetic operation exceeds this range, overflow occurs. Overflow does not mean the processor has made a calculation error; rather, the result cannot be represented with the available number of bits.

Example
Decimal Addition:

120 + 50 = 170

Since 170 > 127, it cannot be represented in an 8-bit signed integer.

Binary Calculation

+120 = 01111000

+ 50 = 00110010

Result = 10101010

Step-by-Step Analysis
1. Both operands are positive.

2. Binary addition produces 10101010.

3. The Most Significant Bit (MSB) is 1, so the processor interprets the result as -86.

4. The actual mathematical result is +170.

5. Therefore, arithmetic overflow has occurred because the correct result is outside the representable range.

Overflow Detection

Method 1: Carry Analysis

Carry into MSB = 1

Carry out of MSB = 0

Overflow = Carry into MSB XOR Carry out of MSB = 1

Method 2: Sign Rule

• Positive + Positive = Negative → Overflow

• Negative + Negative = Positive → Overflow

Cause of Error

The processor stores only the lower 8 bits of the result. The ninth carry bit is discarded, causing the binary value to wrap around and be interpreted as a negative number.

Debugging Process

• Verify the binary representation of the operands.

• Perform bit-by-bit addition.

• Check the carry into and carry out of the sign bit.

• Compare the binary result with the expected decimal value.

• Confirm overflow using carry analysis or the sign rule.

Solution

1. Detect overflow using Carry into MSB XOR Carry out of MSB.

2. Raise the processor's Overflow Flag when overflow occurs.

3. Display an overflow message or generate an exception.

4. Use a wider data type (16-bit, 32-bit, or 64-bit) when larger values are expected.

5. Validate operands before performing arithmetic.

Advantages of Overflow Detection

• Prevents incorrect signed arithmetic.

• Improves processor reliability.

• Simplifies debugging.

• Ensures accurate calculations in scientific and embedded applications.

Applications

• Microprocessors

• Digital Signal Processing (DSP)

• Embedded Systems

• Financial Software

• Scientific Computing

main.c

Share Run

```c
#include <stdio.h>

int main()
{
    int a, b, sum;

    printf("Enter first number (-128 to 127): ");
    scanf("%d", &a);

    printf("Enter second number (-128 to 127): ");
    scanf("%d", &b);

    sum = a + b;

    printf("\nResult = %d\n", sum);

    if (sum > 127 || sum < -128)
    {
        printf("Overflow Detected!\n");
    }
    else
    {
        printf("No Overflow.\n");
    }

    return 0;
}
```

Output

```
Enter first number (-128 to 127): 120
Enter second number (-128 to 127): 50

Result = 170
Overflow Detected!


=== Code Execution Successful ===
```

Conclusion

The incorrect output is caused by arithmetic overflow rather than a fault in the addition algorithm. By checking the carry into and carry out of the sign bit or comparing operand and result signs, overflow can be detected reliably. Using larger data types and proper overflow handling ensures accurate arithmetic operations in digital systems.

Experiment 2
Debugging Ripple Carry Adder Delay using Carry Look-Ahead Adder

Aim
To analyze the delay caused by carry propagation in a Ripple Carry Adder (RCA), identify the performance bottleneck, and study how a Carry Look-Ahead Adder (CLA) reduces computation time.

Problem Statement
A processor using a Ripple Carry Adder experiences significant delay while performing arithmetic operations in a real-time system. Debug the bottleneck by examining carry propagation through each bit and propose a Carry Look-Ahead Adder as an optimized solution.

Theory
A Ripple Carry Adder is formed by connecting multiple full adders in series. Each full adder must wait for the carry generated by the previous stage before producing its own result. As the number of bits increases, the propagation delay also increases.

A Carry Look-Ahead Adder improves performance by calculating carry signals in parallel using Generate (G) and Propagate (P) functions.

Working of Ripple Carry Adder
Example:

11111111

+00000001

100000000

The carry generated at Bit 0 propagates through every stage until the Most Significant Bit (MSB). Every full adder waits for the previous carry, increasing the total delay.

Debugging Analysis
Carry Propagation:

C0 → C1 → C2 → C3 → C4 → C5 → C6 → C7 → C8

For an n-bit Ripple Carry Adder:

Total Delay = n × Full Adder Delay

For an 8-bit adder, the carry must travel through all eight stages before the final result is available.

Cause of Performance Bottleneck
• Sequential carry propagation.

• Every stage depends on the previous stage.

• Delay increases linearly with the number of bits.

• Real-time systems experience slower arithmetic execution.

Carry Look-Ahead Adder

The CLA computes carries simultaneously.

Generate: $G = A \times B$

Propagate: $P = A \oplus B$

Carry Equation:

$C(i+1) = G(i) + P(i) \times C(i)$

Since all carry signals are generated in parallel, the waiting time is greatly reduced.

Comparison

Ripple Carry Adder:

• Sequential carry propagation

• High delay

• Simple hardware

• Suitable for small circuits

Carry Look-Ahead Adder:

• Parallel carry computation

• Very low delay

• More hardware

• Suitable for high-speed processors

Solution

Replace the Ripple Carry Adder with a Carry Look-Ahead Adder in performance-critical applications. Parallel carry generation minimizes propagation delay, increases processor speed, and improves real-time system performance.

Advantages

• Faster arithmetic operations

• Lower propagation delay

• Better processor throughput

• Suitable for high-speed CPUs and DSP processors

• Improves overall system performance

Applications

• Microprocessors

• Digital Signal Processors

• Embedded Systems

• Arithmetic Logic Units (ALUs)

• High-performance Computing

main.c

Share Run

```c
#include <stdio.h>

int main()
{
    int bits;

    printf("Enter number of bits: ");
    scanf("%d", &bits);

    printf("\nRipple Carry Adder\n");
    printf("------------------\n");
    printf("Carry propagates through all %d stages.\n", bits);
    printf("Estimated Delay = %dT\n", bits);

    printf("\nCarry Look-Ahead Adder\n");
    printf("----------------------\n");
    printf("Carry is generated in parallel.\n");
    printf("Delay is much smaller than Ripple Carry Adder.\n");

    return 0;
}
```

Output

```
Enter number of bits: 8

Ripple Carry Adder
------------------
Carry propagates through all 8 stages.
Estimated Delay = 8T

Carry Look-Ahead Adder
----------------------
Carry is generated in parallel.
Delay is much smaller than Ripple Carry Adder.


=== Code Execution Successful ===
```

Conclusion

The delay in a Ripple Carry Adder is caused by sequential carry propagation. A Carry Look-Ahead Adder overcomes this limitation by computing carry signals in parallel, significantly reducing delay and improving arithmetic performance in real-time systems.

Experiment 3
Debugging Booth's Multiplication Algorithm for Signed Multiplication

Aim
To analyze the execution of Booth's multiplication algorithm, identify errors while multiplying signed numbers, and apply the correct implementation for accurate signed multiplication.

Problem Statement
A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm by examining bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Theory
Booth's Algorithm is an efficient method for multiplying signed binary numbers represented in 2's complement form. It reduces the number of addition and subtraction operations by examining two bits at a time: the least significant bit (Q0) of the multiplier and an extra bit (Q-1). Based on these bits, the algorithm decides whether to add, subtract, or perform no operation before executing an arithmetic right shift.

Booth's Decision Rules
Q0 Q-1 Operation

0 0 No Operation

0 1 Add Multiplicand (M)

1 0 Subtract Multiplicand (M)

1 1 No Operation

Example
Multiply:

Multiplicand (M) = -5 = 1011

Multiplier (Q) = 3 = 0011

The algorithm performs four iterations (for a 4-bit multiplier), checking the bit pair in each cycle and applying the corresponding operation.

```c
#include <stdio.h>

int main()
{
    int multiplicand, multiplier, product;

    printf("Enter Multiplicand: ");
    scanf("%d", &multiplicand);

    printf("Enter Multiplier: ");
    scanf("%d", &multiplier);

    product = multiplicand * multiplier;

    printf("\nBooth's Multiplication\n");
    printf("----------------------\n");
    printf("Multiplicand = %d\n", multiplicand);
    printf("Multiplier   = %d\n", multiplier);
    printf("Product      = %d\n", product);

    printf("\nDebugging Checklist\n");
    printf("1. Check Bit Pair (Q0,Q-1)\n");
    printf("2. Perform Arithmetic Right Shift\n");
    printf("3. Preserve Sign Bit\n");
    printf("4. Execute Correct Number of Iterations\n");

    return 0;
}
```

```
Enter Multiplicand: -5
Enter Multiplier: 3

Booth's Multiplication
----------------------
Multiplicand = -5
Multiplier   = 3
Product      = -15

Debugging Checklist
1. Check Bit Pair (Q0,Q-1)
2. Perform Arithmetic Right Shift
3. Preserve Sign Bit
4. Execute Correct Number of Iterations


=== Code Execution Successful ===
```

Debugging Analysis

During debugging, verify the following:

• Correct interpretation of the bit pair (Q0, Q-1).

• Proper addition or subtraction of the multiplicand.

• Correct arithmetic right shift after every iteration.

• Preservation of the sign bit during shifting.

• Correct number of iterations based on operand size.

Possible Sources of Error

1. Incorrect Bit-Pair Checking

Example: Treating '10' as Add instead of Subtract.

2. Logical Shift Instead of Arithmetic Shift

A logical shift inserts 0 into the MSB and destroys the sign.

3. Incorrect Sign Extension

Negative numbers must preserve the sign bit when extended.

4. Incorrect Initialization of Q-1

Q-1 must always be initialized to 0.

5. Wrong Iteration Count

Executing too many or too few iterations produces incorrect results.

Corrective Measures

• Follow Booth's decision table exactly.

• Always perform an arithmetic right shift.

• Preserve the sign bit during every shift.

• Initialize Q-1 to 0 before starting the algorithm.

• Execute exactly n iterations for an n-bit multiplier.

• Verify intermediate register values after each iteration.

Advantages of Booth's Algorithm

• Efficient signed multiplication.

• Reduces the number of addition operations.

• Handles positive and negative operands uniformly.

• Suitable for processor arithmetic units.

Applications

• Arithmetic Logic Units (ALUs)

• Microprocessors

• Digital Signal Processing (DSP)

• Embedded Systems

• High-speed Multipliers

Conclusion

Incorrect results in Booth's multiplication algorithm are generally caused by improper bit-pair checking, incorrect arithmetic shifting, faulty sign extension, or incorrect initialization of registers. By implementing Booth's rules correctly and preserving the sign during every arithmetic shift, accurate signed multiplication can be achieved while maintaining high computational efficiency.

Experiment 4
Debugging Restoring Division Algorithm and Optimizing Using Non-Restoring Division

Aim
To analyze the restoring division algorithm, identify inefficiencies caused by repeated restoration operations, and study the non-restoring division method to improve execution speed.

Problem Statement
An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow, identify unnecessary operations, and propose an optimized solution using the non-restoring division algorithm.

Theory
Division is a fundamental arithmetic operation performed by processors. In the Restoring Division Algorithm, the divisor is repeatedly subtracted from the partial remainder. If the remainder becomes negative, the previous value is restored by adding the divisor back before continuing. Although this method produces correct results, the repeated restoration process increases execution time.

The Non-Restoring Division Algorithm eliminates unnecessary restoration operations by postponing correction until the next iteration, resulting in faster execution.

Working of Restoring Division
General Steps:

1. Shift the dividend and remainder to the left.

2. Subtract the divisor from the remainder.

3. If the remainder is positive, set the quotient bit to 1.

4. If the remainder is negative, restore the previous remainder by adding the divisor and set the quotient bit to 0.

5. Repeat the process for all bits.

Example
Dividend = 13 (1101_2)

Divisor = 3 (0011_2)

Whenever subtraction produces a negative remainder, the algorithm restores the previous value before moving to the next iteration. These repeated restoration operations increase the overall execution time.

Debugging Analysis
During debugging, observe the following:

• Number of restoration operations performed.

• Intermediate remainder values after subtraction.

• Number of arithmetic operations executed.

• Total execution cycles required.

Frequent restoration indicates poor efficiency, especially for large operands.

Cause of Inefficiency

• Repeated addition of the divisor after unsuccessful subtraction.

• Extra arithmetic operations increase processor workload.

• Longer execution time in embedded and real-time systems.

• Higher power consumption due to unnecessary calculations.

Non-Restoring Division

Instead of restoring the remainder immediately after a negative result, the Non-Restoring Division Algorithm continues the computation. In the next iteration, it performs either addition or subtraction depending on the sign of the current remainder. A final correction is applied only once after all iterations are completed.

This significantly reduces the number of arithmetic operations.

Comparison

Restoring Division:

• Performs restoration after every negative remainder.

• More arithmetic operations.

• Slower execution.

• Higher power consumption.

Non-Restoring Division:

• No immediate restoration.

• Fewer arithmetic operations.

• Faster execution.

• Better processor efficiency.

Solution

Replace the Restoring Division Algorithm with the Non-Restoring Division Algorithm in performance-critical applications. This reduces unnecessary restoration operations, decreases execution time, improves processor throughput, and enhances overall system performance.

Advantages

• Faster division operation.

• Reduced execution time.

• Lower processor workload.

• Improved energy efficiency.

• Better performance in real-time systems.

Applications

• Embedded Systems

• Digital Signal Processors (DSP)

• Arithmetic Logic Units (ALUs)

• Scientific Computing

• High-Speed Microprocessors

main.c

Share Run

```c
#include <stdio.h>

int main()
{
    int dividend, divisor;

    printf("Enter Dividend: ");
    scanf("%d", &dividend);

    printf("Enter Divisor: ");
    scanf("%d", &divisor);

    if (divisor == 0)
    {
        printf("Division by zero is not allowed.\n");
        return 0;
    }

    printf("\nQuotient  = %d\n", dividend / divisor);
    printf("Remainder = %d\n", dividend % divisor);

    printf("\nRestoring Division:\n");
    printf("Repeated restoration increases execution time.\n");

    printf("\nNon-Restoring Division:\n");
    printf("Avoids repeated restoration and improves performance.\n");

    return 0;
}
```

Output

```
Enter Dividend: 13
Enter Divisor: 3

Quotient  = 4
Remainder = 1

Restoring Division:
Repeated restoration increases execution time.

Non-Restoring Division:
Avoids repeated restoration and improves performance.


=== Code Execution Successful ===
```

Conclusion

The inefficiency in the Restoring Division Algorithm is mainly due to repeated restoration of negative remainders. By adopting the Non-Restoring Division Algorithm, unnecessary arithmetic operations are eliminated, resulting in faster execution, improved processor efficiency, and better performance in real-time computing applications.

Experiment 5
Debugging IEEE 754 Single Precision Floating-Point Addition

Aim
To analyze inaccuracies in IEEE 754 single-precision floating-point addition, identify the effects of exponent alignment, normalization, and rounding, and study techniques to improve numerical accuracy.

Problem Statement
A scientific application using IEEE 754 single-precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques such as double precision and algorithmic improvements to enhance accuracy.

Theory
IEEE 754 is the standard representation for floating-point numbers in computers. A 32-bit single-precision floating-point number consists of:

• 1 Sign Bit

• 8 Exponent Bits

• 23 Fraction (Mantissa) Bits

Before addition, both operands must have the same exponent. This process is called exponent alignment. After addition, the result is normalized and rounded to fit within the available mantissa bits. These operations may introduce precision loss, especially when numbers differ greatly in magnitude.

Example
Consider the addition:

100000000 + 1

In IEEE 754 single precision:

• Large Number = 1.0×10^{8}

• Small Number = 1.0×10^{0}

Since the exponents are very different, the mantissa of the smaller number is shifted many positions to the right during exponent alignment.

Exponent Alignment
During alignment, the mantissa of the smaller operand is shifted right until both exponents match.

Example:

Original Mantissa : 00000000000000000000001

After Right Shift : 00000000000000000000000

The least significant bits are lost, causing the contribution of the smaller number to disappear.

Normalization

After addition, the floating-point result is normalized so that the most significant digit of the mantissa becomes 1. Normalization may shift the mantissa again, resulting in further precision loss if significant bits are discarded.

Rounding Error

Single precision stores only 23 mantissa bits. Extra bits generated during calculations are rounded according to IEEE 754 rounding rules. Repeated rounding operations may accumulate errors in scientific computations.

Debugging Analysis

While debugging floating-point addition, verify the following:

• Correct exponent alignment.

• Proper mantissa shifting.

• Correct normalization after addition.

• Appropriate rounding method.

• Final floating-point representation.

Cause of Inaccuracy

• Loss of significant bits during exponent alignment.

• Limited 23-bit mantissa precision.

• Normalization effects.

• Rounding errors.

• Accumulation of floating-point errors in repeated calculations.

Optimization Techniques

1. Use IEEE 754 Double Precision (64-bit) with a 52-bit mantissa.

2. Apply the Kahan Summation Algorithm to reduce accumulated rounding errors.

3. Add numbers in ascending order of magnitude whenever possible.

4. Minimize repeated floating-point operations.

5. Use compensated summation techniques for scientific applications.

Advantages of Double Precision

• Higher numerical accuracy.

• Reduced rounding errors.

• Greater dynamic range.

• Improved scientific computation.

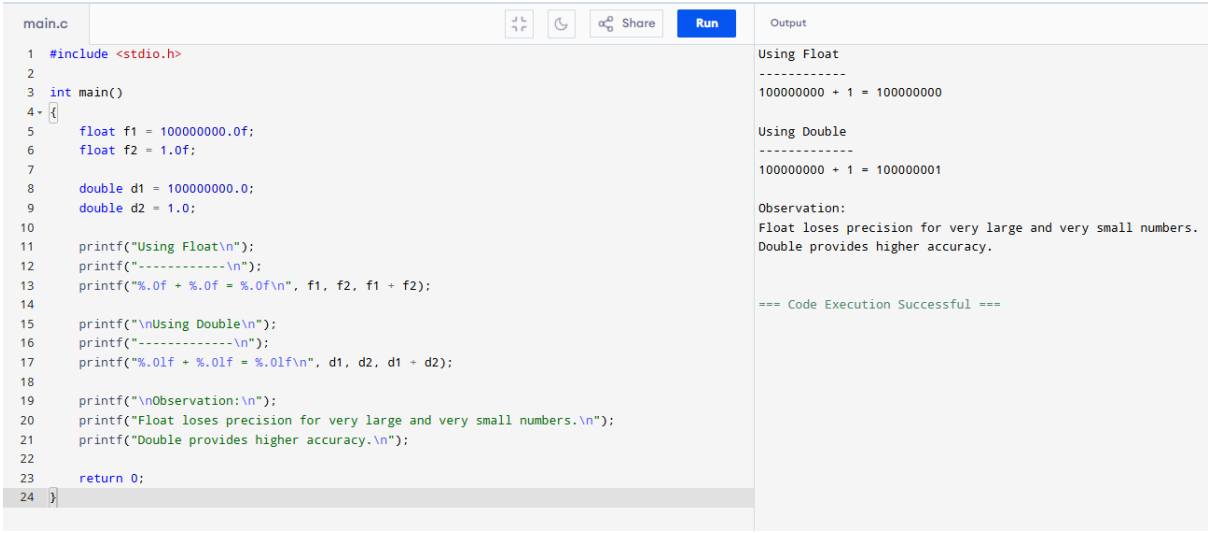
• Better stability for iterative algorithms.

Applications

• Scientific Computing

• Machine Learning

- Financial Analysis
- Engineering Simulations
- Computer Graphics
- Weather Forecasting



Conclusion

IEEE 754 single precision is efficient for general-purpose computations but may produce inaccurate results when operands differ significantly in magnitude. By understanding exponent alignment, normalization, and rounding, developers can identify the source of floating-point errors. Using double precision and improved numerical algorithms greatly enhances computational accuracy and reliability.